

Beamline Voltage Optimization

A Hands-On Activity with SIMION and Optuna

MIT–Medellín Hackathon

1 The goal

You are given a small *ion beamline* simulated in SIMION. A beam of ions is launched at one end, and your job is to steer and focus it so that as many ions as possible reach a detector at the other end. How well the beamline does this is called its **transmission**:

$$\text{Transmission} = \frac{\text{ions that reach the detector}}{\text{ions launched (500)}} \times 100 \, \%.$$

The catch: the transmission depends on the *voltages* we put on the electrodes, and there are many electrodes. Trying combinations by hand is hopeless. Instead, you will let an optimization library called **Optuna** search for the best voltages automatically, using SIMION as the “referee” that scores every guess. **Your final answer is the set of voltages that maximizes transmission.**

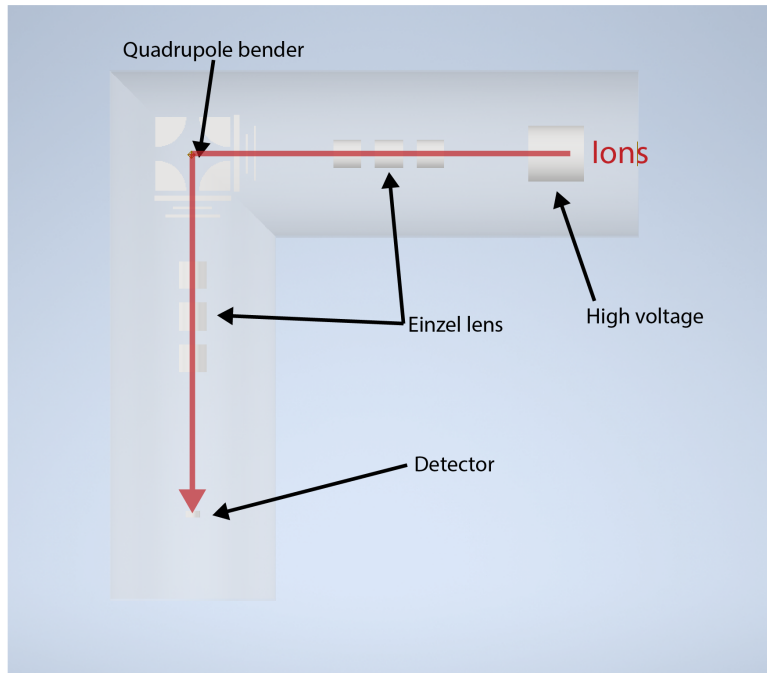


Figure 1: The beamline. Ions are born inside the *high-voltage* cage on the right and travel left through two *Einzel lenses* (which focus the beam). The *quadrupole bender* turns the beam 90° at the corner, and the ions then travel down to the *detector*. In simulation coordinates the beam starts at (395, 75, 77) mm moving along $-x$, is bent into $+z$, and is collected in a small box around (76, 76, 405) mm.

2 The simulation files

All of these files live together in one *project folder*, the same folder you keep `optimizer.py` in. The optimizer finds that folder automatically and uses *relative* paths, so it does not matter where the folder sits on your particular computer (the drive letter and the path will be different on every machine). Here is what each kind of file is for.

2.1 STL files: the geometry

The `electrode_1.stl`, `electrode_2.stl`, ... files are 3D models (exported from CAD) of the *shape* of each metal electrode: the high-voltage cage, the pipe, the lens rings, the bender, the plates, and the detector. They describe *where the metal is*, but not what voltage it is at. SIMION reads these shapes to build the simulation.

2.2 The .PA# and .PA0 files: the electric fields

A *potential array* (PA) is SIMION’s grid describing the electric setup in space. There are three kinds, and only the first one is given to you:

- `electrode_.PA#`: the *definition* file. It records the geometry (which grid points are metal, and which electrode number each belongs to) but **not** the electric field yet. This is the single file you start from. *You generate the other two from it* by “refining” (see Step 0 in Section 4).
- `electrode_.PA1`, `electrode_.PA2`, ..., `electrode_.PA19`: one *solved* field per electrode, computed with that electrode at unit voltage and all others grounded. Solving these is the slow step, but it only has to be done once.
- `electrode_.PA0`: the combined “*fast-adjust*” array. Because fields add up linearly, SIMION can take your chosen voltage for every electrode and *instantly* build the total field by scaling and summing the per-electrode arrays, with no slow re-solving. This is what lets the optimizer test thousands of voltage combinations quickly.

The `.PA1...PA19` and `.PA0` files are large (hundreds of MB each), which is why they are *not* shipped to you; you create them once on your own machine by refining the small `.PA#`.

2.3 The .fly2 file: the ion beam

`SimpleSetUp.fly2` defines the particles we shoot in: **500** singly-charged silicon ions ($^{28}\text{Si}^+$, mass 28 amu, charge +1), with a kinetic energy of about 15 eV, launched from position (395, 75, 77) mm in a 15° cone aimed along the $-x$ direction. Every run flies this same beam, so any change in transmission is due to the *voltages*, not the beam.

2.4 The .iob file: the workbench

`SimpleSetUp.iob` is the SIMION “workbench”: it ties everything together (the geometry, the potential arrays, the beam, and the recording settings) into one runnable simulation. When we tell SIMION to “fly”, this is the file we point it at.

3 Which voltages are fixed, and which you optimize

Not every electrode is a free knob. Some *must* stay at a set voltage because of the role they play (file `ElectrodesMaps.txt`):

- **High-voltage cage (+500 V).** This is the charged enclosure where the ions are created. Its potential sets the starting energy of the beam, so it is a fixed property of the source, not a tuning knob.
- **Grounding pipe and ground plates (0 V).** These are the reference (“earth”) electrodes and electrostatic shields. Ground *is* the zero of our voltage scale; changing it would not steer the beam, it would just move the reference.
- **Detector (−2000 V).** The detector is held strongly negative so it attracts and collects the positive ions. It is the measuring device at the end of the line, so its voltage is fixed too.

Everything that *shapes* the beam (the lenses, the bender, and the steering plates) is left free. **Those are the voltages you optimize.** In `ElectrodesMaps.txt` they are exactly the electrodes that have *no* value written next to them. There are eight of them, so this is an 8-dimensional search.

Table 1: The electrodes of the beamline.

Electrode(s)	Component	Voltage
1	High-voltage cage (ion source)	+500 V (fixed)
2	Grounding pipe	0 V (fixed)
3	Einzel lens 1 (biased element)	optimize
4, 5	Einzel lens 1 (grounded elements)	0 V (fixed)
6	Einzel lens 2 (biased element)	optimize
7, 8	Einzel lens 2 (grounded elements)	0 V (fixed)
9–12	Quadrupole bender (four parts)	optimize
13, 14	Ground plates	0 V (fixed)
15	Voltage / steering plate 1	optimize
16, 17	Ground plates	0 V (fixed)
18	Voltage / steering plate 2	optimize
19	Detector	−2000 V (fixed)

So the optimizer is free to choose voltages for electrodes **3, 6, 9, 10, 11, 12, 15, and 18**; all the others are pinned to the values above on every single run.

Keep the voltages physical. For each free electrode you must give the optimizer a *range* of voltages to search within. Choose those ranges to match what real lab power supplies can actually produce; they cannot output millions of volts. As a rule of thumb, ± 500 V is comfortably available, and ± 1000 V **is the maximum you should allow**. A voltage the power supply cannot deliver is useless, even if the simulation “likes” it, so do not let the search wander beyond that. (The fixed +500 V source and −2000 V detector run on their own dedicated high-voltage supplies, which is why those two are allowed to sit outside the ± 1000 V range.)

4 Running SIMION from the terminal

Once SIMION is set up, almost everything happens through a few command-line instructions. You run **Step 0** **once** at the very beginning; the optimizer then runs **Steps 1 and 2 automatically** for every guess it makes (and you can also run those two by hand to see what they do).

Make sure “simion” is runnable first. Every command below starts with the word `simion`. For your terminal to understand it, your computer has to know where SIMION is installed, and

that folder is different on every machine. If you type `simion` and see an error like “`simion is not recognized`” (Windows) or “`command not found`”, pick one of these:

- **Add SIMION to your PATH.** Add the SIMION install folder (the one containing `simion.exe`, normally `C:\Program Files\SIMION-8.1`) to your system’s *Path* environment variable, then open a *new* terminal. Now plain `simion` works everywhere, exactly as written below.
- **Or write the full path to the program** in every command, in place of the word `simion`. In PowerShell you put the `&` call operator in front of the quoted path, for example:

```
& "C:\Program Files\SIMION-8.1\simion.exe" --nogui refine "electrode_.PA#"
```

(The Python optimizer already knows where SIMION is from the `SIMION_INSTALL_DIR` setting in its `CONFIG`, so this only matters for commands *you* type by hand.)

The SIMION manual. The easiest reference is the PDF included with this activity, `simion8manual.pdf`. It documents an older SIMION version, but the command-line tools are the same, and they are all described in **Appendix M (Command Line Interface)**; look there for every option of `refine`, `fastadj` and `fly`.

Your installed SIMION also ships the newer, complete manual as `simion.chm` in the SIMION folder (the one that holds `simion.exe`); search it for “command line”. Note that `.chm` is a Windows Help file, and Windows often **blocks** it (via Smart App Control, or the file’s “Mark of the Web”), so it may open blank or say “Navigation to the webpage was canceled”. To allow it: right-click `simion.chm` → *Properties* → tick *Unblock* → *OK*. If Smart App Control still blocks it, copy it to a local folder such as your Desktop and open it from there.

Step 0: refine the potential array (refine), do this once. You are given only the geometry file `electrode_.PA#`. “Refining” is the step where SIMION solves the electric field for the geometry and writes out the heavy field files: `electrode_.PA0` plus the per-electrode arrays `electrode_.PA1...PA19`. Run it once from the project folder:

```
simion --nogui refine "electrode_.PA#"
```

This can take a few minutes and some memory; that is normal, because you are solving a field on a grid of millions of points. You only ever do it once; after that the field files exist and flying is fast. If you change the *geometry*, refine again; if you only change *voltages*, you do not.

Step 1: set the voltages (fastadj). The `fastadj` sub-command writes a voltage onto every electrode of the fast-adjust `.PA0` array. It takes the array file followed by a comma-separated list of `electrode=voltage` pairs. *Assembling this command is your Task 1b in `optimizer.py`.*

Step 2: fly the ions and see where they land (fly). The `fly` sub-command sends the 500 ions through the field defined by the workbench file and prints where each one lands. *Assembling this command is your Task 1a in `optimizer.py`.*

You build both commands yourself, from the pieces in the table below and the SIMION manual (`simion8manual.pdf`, Appendix M; see the note above). Here is what each piece does:

After the flight, SIMION prints one line of the form `xyz(76, 75, 405)mm` for each ion that reaches the detector. The optimizer counts those lines; that count is the transmission score.

Table 2: What each part of the SIMION command means.

Part of the command	What it does
<code>simion</code>	Launches the SIMION program.
<code>--nogui</code>	Runs it <i>headless</i> (no graphical window) so a script can drive it automatically.
<code>fly</code>	The sub-command that actually simulates (“flies”) the ions through the field.
<code>fastadj</code>	The sub-command that instantly sets electrode voltages, using the fast-adjust <code>.PA0</code> array.
<code>"electrode_.PA0"</code>	The potential-array file whose voltages we are setting. (Quoted because the folder path contains a space.)
<code>1=500,2=0,...</code>	The list of electrode=voltage assignments. We set <i>every</i> electrode each time so each run is fully reproducible.
<code>--recording-output=out.txt</code>	The file SIMION writes its recorded data to.
<code>--retain-trajectories=0</code>	Do <i>not</i> keep every ion’s full flight path in memory, since we only need where each ion ends up. Faster and lighter.
<code>--restore-potential=0</code>	Do <i>not</i> undo the voltages after the run; leave the field we just set in place.
<code>--programs=0</code>	Run the batch flight without extra user-program passes, keeping each run lean.
<code>"SimpleSetUp.iob"</code>	The workbench (geometry + beam + recording) we are flying.

5 What you are going to do

Make sure you have already done Step 0 (refined `electrode_.PA#`) so the field files exist. Then you run a single Python program, `optimizer.py`:

```
python optimizer.py
```

Behind the scenes it repeats this loop (a method called *Bayesian optimization*):

1. **Optuna** **proposes** a voltage for each of the 8 free electrodes (within ranges you can edit at the top of the file).
2. The script **sets** those voltages with `fastadj`.
3. SIMION **flies** the 500 ions with `fly`.
4. The script **counts** how many ions reached the detector; that count is the score.
5. Optuna **learns** from the score and proposes a smarter set of voltages next time.

After all the trials, the best-scoring set of voltages is your result. Results are also saved to `beamline_results.csv` so you can plot and inspect them.

Your task. Use `optimizer.py` to find the voltages on electrodes 3, 6, 9–12, 15, and 18 that give the *highest transmission*. Report those voltages and the transmission they achieve.

Things to experiment with:

- Change `N_TRIALS`: does a longer search find a better answer?
- Widen or narrow the voltage ranges in `OPTIMIZE` (but keep them within the ± 1000 V physical limit).
- Switch `OBJECTIVE` from "hits" to "spread" to aim for the *tightest* beam at the detector instead of the most ions.
- Give the optimizer a good `STARTING_POINT` to warm-start from.